

DE и AI-плагины

- Основной редактор: Cursor как основной IDE для кода (замена VS Code, совместим с его экосистемой расширений).
- Дополнительно: VS Code с GitHub Copilot и/или Codeium для проектов, где Cursor не обязателен.
- Полезные штуки вокруг Cursor: ориентируюсь на экосистему из awesome-cursor (CursorLens, specstory, Cursor Stats, cursor-tools и т.п., как инструменты для логирования генераций, статистики и улучшения Composer-флоу).

Модели и под какие задачи

Обычно использую несколько классов моделей, переключаясь по задаче:

- Для генерации и рефакторинга кода в IDE: модели, подключаемые в Cursor (по сути, «AI-native» редактор, умеющий работать с несколькими файлами сразу в Composer-режиме для проектного рефакторинга).
- Для самостоятельных агентов / бэкенда: модели, подключённые через MCP-серверы (см. ниже) — LLM с поддержкой function calling, чтобы дергать инструменты и внешние API.
- Для «офисных» задач: более дешёвые/быстрые модели (через тот же MCP или отдельные интеграции) для суммаризации, черновиков писем, генерации документации и инструкций.

Какие MCP использую

MCP (Model Context Protocol) использую как стандартный способ подключать LLM к внешним инструментам и данным:

- MCP-серверы, дающие доступ к API: собственные маленькие Python-серверы (по типу FastMCP) для

REST-API, внутренних сервисов, Jira/YouTrack, GitLab и т.п.

- MCP для IDE/кода: решения из экосистемы вроде «Cursor MCP» и похожие — мост между десктоп-клиентом и редактором, чтобы LLM мог работать с проектом, задачами, документацией.
- MCP-сервера ресурсов: серверы, которые отдают контекст как «resources» (логика: LLM может читать данные без сайд-эффектов — документация, конфиги, справочники).

Архитектурно придерживаюсь разделения на Hosts, Clients, Servers, Tools/Resources/Prompts, как в описании MCP: host (IDE или агент), client как адаптер, server как поставщик tools/resources/prompts.

cursorrules / системные инструкции и полезные правила

Да, использую системные инструкции и project rules (аналог cursorrules) практически во всех проектах, особенно где много шаблонного кода или строгий стиль.

Основные практики, которые считаю самыми полезными:

- Разделение уровней правил:
 - глобальные user rules для общих привычек (языки, стиль комментариев, приоритет тестов);
 - project rules в `.cursor/rules/*.mdc`, версионизируемые в git для конкретного репо.
- Краткость и фокус: держать правила компактными (идеально <500 строк), разбивать на логические блоки (frontend, backend, tests, infra) вместо одного полотна.
- Чёткие требования к стилю кода:

- форматирование (prettier/black/go fmt, не менять вручную);
- принцип «не ломать существующие тесты»;
- предпочтения по паттернам (DI, CQRS, Hexagonal, DRY против дублирования).
- Явные указания по работе с проектом:
 - где хранить конфиги;
 - как именовать переменные/файлы;
 - какие части кода лучше не трогать без отдельного запроса.
- Инструкция «сначала уточни задачу»: правило, чтобы AI сначала кратко пересказал, что понял, и предложил план, прежде чем что-то менять в коде — это сильно экономит время на откаты.

Из практических примеров правил, которые реально помогают:

- «Не удаляй существующие проверки/валидации без прямого указания».
- «При изменении публичного API — всегда обновляй документацию и примеры использования».
- «Если задача большая — сначала предложи поэтапный план коммитов/PR».
- «Всегда показывай diff/изменённые участки, а не весь файл, если не просили иное».